

To illustrate this concept, let's use the `DimmerSwitch` class. In the following example, we'll instantiate two `DimmerSwitch` objects, then walk through what happens when we raise the brightness level of these objects by calling the `raiseLevel()` method with each.

The code of the method we're calling is:

```
def raiseLevel(self):
    if self.brightness < 10:
        self.brightness = self.brightness + 1
```

[Listing 3-3](#) shows some example test code for two `DimmerSwitch` objects.

File: `OO_DimmerSwitch_Model2_Method_Calls.py`

```
# Create two DimmerSwitch objects
oDimmer1 = DimmerSwitch('Dimmer1')
oDimmer2 = DimmerSwitch('Dimmer2')

# Tell oDimmer1 to raise its level
oDimmer1.raiseLevel()

# Tell oDimmer2 to raise its level
oDimmer2.raiseLevel()
```

Listing 3-3: Calling the same method on different `DimmerSwitch` objects

In this listing, we first instantiate two `DimmerSwitch` objects. Then we have two calls to the `raiseLevel()` method: first we call it with `oDimmer1`, then we call the same method using `oDimmer2`.

[Figure 3-5](#) shows the result of running the test code in [Listing 3-3](#) in Python Tutor, with execution stopped while making the first call to `raiseLevel()`.